COMPRESSED SENSING PATHWAYS FOR LEARNING MATHEMATICS THROUGH THE MACHINE

DR. JIMMIE ADRIAZOLA, SCHOOL OF MATHEMATICAL AND STATISTICAL SCIENCES, ASU

Big Picture Idea

At its heart, this extension of *Learning Mathematics Through the Machine* asks a sharper question than before:

If the correct mathematical law hides inside a very large library of candidate formulas, can sparsity reveal it?

Compressed sensing gives a serious mathematical framework for answering this question. Instead of prescribing the correct symbolic form in advance, we build a wide dictionary of candidate atoms—polynomials, orthogonal polynomials, logarithms, exponentials, trigonometric terms, rational pieces, recurrence terms, or tensor-product monomials—and then ask an optimization method to recover only the very small number of atoms truly needed to explain the data. The central hope is that mathematical truth, at least in many of the examples accessible to our students, is not merely accurate but also *parsimonious*. In other words, the truth often admits a representation with only a few nonzero coefficients when viewed in the right basis. This is exactly the regime where compressed sensing thrives [5, 10, 12].

This note is a technical plan for turning that philosophical idea into a concrete undergraduate research program. The goal is not merely to mention sparsity, but to train students to use the optimization methodologies that applied mathematicians actually take seriously: QR/SVD-based least squares, greedy sparse recovery, proximal gradient methods, accelerated methods, alternating direction methods of multipliers, block and coordinate descent, iteratively reweighted schemes, primal–dual methods for analysis sparsity, and low-rank relaxations where appropriate [1–3, 7, 9, 14, 15, 17]. The world is a funny little machine: once the right linear algebra is in place, even a stubborn symbolic problem starts to look like an inverse problem in disguise.

Project Context and Impact

The original LSSR pipeline (Least-Squares → Sparsify → Refine) already contains the seed of compressed sensing. The least-squares stage produces a dense approximation, the sparsification stage selects a support, and the refinement stage debiases that support by re-solving the regression problem on the reduced subspace. In compressed sensing language, this is support identification followed by oracle refitting. That observation is not cosmetic. It means the original project can be extended in a mathematically principled way to settings where the hypothesis class is much wider, the dictionary is redundant, and the need for structure is more acute. Once the hypothesis library becomes large, naive thresholding may still work on easy problems, but a more systematic sparse recovery toolbox becomes essential [4, 12].

Pedagogically, compressed sensing is ideal for an undergraduate team because it sits at the intersection of several core subjects they are already learning. It uses linear algebra to represent hypotheses, numerical analysis to generate reliable data, optimization to recover coefficients, statistics to tune regularization and validate support recovery, and proof-based mathematics to turn a floating-point pattern into an actual theorem. Better still, many of the most useful algorithms are computationally light enough to run on ordinary laptops, especially in the small-to-medium regimes relevant to the proposed warm-up problems.

Our aim is therefore twofold. First, we want students to understand compressed sensing as a mathematical theory of parsimony: why sparse models can be recovered from limited or noisy information, when they fail, and how matrix conditioning and coherence control that story. Second, we want students to become competent scientific programmers who can implement and compare serious solvers rather than merely call a black-box package and hope the linear algebra gods are in a good mood that day.

The Core Mathematical Model

Let Ω denote an input domain and let $u : \Omega \to \mathbb{R}$ be the mathematical quantity we wish to learn from numerical data. We assume that we can generate reliable observations at sample points $z_1, \ldots, z_N \in \Omega$ through a numerical oracle. For example, $u(z_k)$ may be computed by quadrature, by solving an ODE, by evaluating a finite-difference approximation, or by counting combinatorial or arithmetic objects.

Fix a dictionary of candidate atoms

$$\Phi_M = \{\phi_1, \phi_2, \dots, \phi_M\}, \qquad \phi_j : \Omega \to \mathbb{R}.$$

We then form the design matrix $A \in \mathbb{R}^{N \times M}$ by

$$A_{kj} = \phi_j(z_k), \qquad 1 \le k \le N,\ 1 \le j \le M,$$

and the observation vector $y \in \mathbb{R}^N$ by

$$y_k = u(z_k) + e_k,$$

where e_k collects all numerical and modeling error.

The synthesis sparsity model assumes that the target admits an expansion of the form

$$u(z) \approx \sum_{j=1}^{M} c_j^\natural \phi_j(z),$$

with coefficient vector $c^\natural \in \mathbb{R}^M$ that is either exactly sparse or well approximated by a sparse vector. In matrix form, this becomes

$$y = Ac^\natural + e.$$

If $\|c^\natural\|_0 = s \ll M$, then the law is represented by only s active atoms, and sparse recovery becomes a natural objective.

Definition 1. *For $c \in \mathbb{R}^M$, the support of c is*

$$\operatorname{supp}(c) = \{j \in \{1, \dots, M\} : c_j \neq 0\}.$$

The vector c is called s-sparse if $\|c\|_0 \le s$.

In practice, many targets are not exactly sparse but *compressible*. This means that their coefficients decay rapidly after sorting by magnitude. A standard quantity measuring this is the best s-term error

$$\sigma_s(c^\natural)_1 = \inf\left\{\|c^\natural - z\|_1 : z \in \mathbb{R}^M,\ \|z\|_0 \le s\right\}.$$

When $\sigma_s(c^\natural)_1$ is small, one should not expect exact recovery, but one may still expect accurate and interpretable approximate recovery.

Matrix diagnostics that matter. Compressed sensing is not magic dust sprinkled onto an arbitrary matrix. The quality of sparse recovery depends strongly on the geometry of the columns of A. In every student project, the following diagnostics should be computed and discussed.

(1) **Column normalization.** Replace each nonzero column a_j of A by $a_j/\|a_j\|_2$. Without this step, regularization unfairly favors columns with large Euclidean norm.
(2) **Mutual coherence.** For normalized columns,

$$\mu(A) = \max_{i \neq j} |\langle a_i, a_j \rangle|.$$

Small coherence is desirable because highly correlated atoms are hard to distinguish.
(3) **Conditioning of selected submatrices.** For a support set S, students should compute $\operatorname{cond}(A_S)$ or the smallest singular value of A_S. Sparse recovery may identify the right support but still produce unstable coefficients if A_S is ill-conditioned.
(4) **Residual correlation plots.** For greedy methods, the quantities $|\langle a_j, r \rangle|$ reveal which atoms compete for the current residual.
(5) **Support stability under resampling.** Repeat the experiment across random training sets. A support that changes wildly is not yet mathematical evidence; it is merely numerical weather.

Two classical recovery conditions will guide the team conceptually.

Definition 2. *The matrix A satisfies the s-restricted isometry property if there exists $\delta_s \in (0,1)$ such that for every s-sparse vector x,*

$$(1 - \delta_s)\|x\|_2^2 \le \|Ax\|_2^2 \le (1 + \delta_s)\|x\|_2^2.$$

Definition 3. *The matrix A satisfies the robust null-space property of order s if there exist constants $\rho \in (0,1)$ and $\tau > 0$ such that for every vector $h \in \mathbb{R}^M$ and every set $S \subset \{1, \dots, M\}$ with $|S| \le s$,*

$$\|h_S\|_1 \le \rho \|h_{S^c}\|_1 + \tau \|Ah\|_2.$$

These notions are central in the theory because they imply stable recovery for ℓ^1 methods [12]. In undergraduate experiments we generally will not prove that a matrix satisfies RIP, but we can design dictionaries and sampling strategies that move us in the right direction by lowering coherence and improving conditioning.

Two benchmark recovery statements students should know. The first benchmark statement is the classical mutual-coherence heuristic. If the columns of A are normalized and the true coefficient vector $c^\natural$ is s-sparse, then a sufficient condition of the form

$$s < \frac{1}{2}\left(1 + \frac{1}{\mu(A)}\right)$$

ensures uniqueness of the sparse representation and supports exact recovery guarantees for methods such as OMP and basis pursuit in the noiseless setting [12, 17]. This bound is not sharp in general, but it is extraordinarily useful pedagogically because students can compute $\mu(A)$ directly and see how basis choice changes the sparse recovery landscape.

The second benchmark statement is the standard stable recovery estimate for ℓ^1 methods. If A satisfies a robust null-space property of order s and the data obey $y = Ac^\natural + e$ with $\|e\|_2 \le \varepsilon$, then a solution $c^\sharp$ of basis-pursuit denoising satisfies an error bound of the form

$$\|c^\sharp - c^\natural\|_2 \le C_0 \frac{\sigma_s(c^\natural)_1}{\sqrt{s}} + C_1\varepsilon,$$

for constants $C_0, C_1 > 0$ depending only on the recovery constants of the matrix [12]. This inequality cleanly separates two effects: model mismatch through the compressibility term $\sigma_s(c^\natural)_1$, and numerical error through ε. It is the right conceptual lens for explaining why some problems yield exact symbolic recovery while others yield only excellent sparse approximations.

For randomized incoherent matrices, one also expects a sample complexity of the rough form

$$N \ge C\, s \log(M/s),$$

which is the famous compressed sensing miracle. In our symbolic-learning setting the matrix is typically far from random, so this estimate should be treated as a guiding heuristic rather than a theorem to quote recklessly. Still, it helps students appreciate that sparse structure may drastically reduce the amount of data needed for reliable discovery.

Why monomial Vandermonde matrices are a bit mischievous. In the original warm-up problem, the design matrix is Vandermonde in the monomial basis. This is pedagogically natural but numerically dangerous. If one uses columns

$$1, r, r^2, \dots, r^m,$$

on an interval such as $[1, 5]$, then nearby powers are strongly correlated, the columns vary wildly in scale, and $A^\top A$ may become very ill-conditioned. This is one reason serious computations should avoid solving least squares by explicitly forming $(A^\top A)^{-1}A^\top y$. Instead, students should use QR factorization, SVD, or iterative least-squares solvers such as LSQR, and they should routinely rescale the domain to $[-1, 1]$ and compare monomials against orthogonal polynomial bases.

For polynomial discovery, the most important dictionary options are:

- **Monomials:** simplest conceptually, but often the worst conditioned.
- **Legendre polynomials:** orthogonal on $[-1, 1]$ with respect to uniform measure; excellent when samples are approximately uniform after rescaling.
- **Chebyshev polynomials:** extremely useful for approximation and numerically stable evaluation; especially good when interpolation or near-minimax behavior matters.
- **Weighted orthogonal bases:** useful when the sampling distribution is not uniform or when a natural weight appears in the mathematics.

This basis choice is already a compressed sensing decision because it changes the geometry of the sensing matrix and can mean the difference between clean sparse recovery and coefficient soup.

Why LSSR Already Lives in the Compressed Sensing Neighborhood

Let $A \in \mathbb{R}^{N\times M}$ and $y \in \mathbb{R}^N$ be as above. The LSSR pipeline may be written schematically as follows.

(1) Solve the dense least-squares problem

$$c^{\mathrm{lsq}} = \operatorname*{argmin}_{c\in\mathbb{R}^M} \frac{1}{2}\|Ac - y\|_2^2.$$

(2) Threshold the entries of c^{lsq} to produce a sparse support estimate S.

(3) Refit on that support:

$$c^{\mathrm{refit}} = \operatorname{argmin}\left\{\frac{1}{2}\|Ac - y\|_2^2 : \operatorname{supp}(c) \subseteq S\right\}.$$

This is a genuine sparse recovery strategy. The thresholding stage identifies a candidate support, while the refit stage performs the classical debiasing step used throughout sparse approximation and compressed sensing.

Proposition 1. *Suppose $S \subset \{1, \dots, M\}$ and A_S has full column rank. Then the constrained least-squares problem*

$$\min \left\{ \frac{1}{2}\|Ac - y\|_2^2 : \operatorname{supp}(c) \subseteq S \right\}$$

has the unique solution given by

$$(c^{\text{oracle}})_S = (A_S^\top A_S)^{-1} A_S^\top y, \qquad (c^{\text{oracle}})_{S^c} = 0.$$

Proof. Restricting to vectors supported on S reduces the problem to ordinary least squares in the variables indexed by S. Since A_S has full column rank, the normal equations on that restricted subspace have a unique solution. □

The real issue, of course, is not how to refit once the support is known. The issue is support identification. This is precisely where the full compressed sensing toolbox becomes useful.

Compressed Sensing Options the Team Can Explore

The purpose of this section is to give a mathematically organized menu of sparse recovery models. Some are immediately appropriate for the undergraduate team; others are better treated as advanced tracks once the baseline infrastructure is stable.

Option A: ℓ^0-constrained sparse regression. The most direct formulation is

$$\min_{c \in \mathbb{R}^M} \frac{1}{2}\|Ac - y\|_2^2 \qquad \text{subject to} \qquad \|c\|_0 \le s.$$

A penalized variant is

$$\min_{c \in \mathbb{R}^M} \frac{1}{2}\|Ac - y\|_2^2 + \lambda \|c\|_0.$$

This model captures the exact spirit of symbolic discovery: use as few atoms as possible. Unfortunately, the problem is combinatorial. Nevertheless, several serious algorithms approximate it effectively when the solution is genuinely sparse and the matrix is not too coherent [2, 12, 15, 17].
Recommended algorithms.

- Orthogonal Matching Pursuit (OMP).
- Compressive Sampling Matching Pursuit (CoSaMP).
- Iterative Hard Thresholding (IHT).
- Hard Thresholding Pursuit (HTP).

These methods are particularly good pedagogically because they make the notion of support explicit. They also connect beautifully to the original LSSR philosophy.

Algorithm (OMP): Orthogonal Matching Pursuit
INPUT: normalized design matrix A, data y, target sparsity $s_{\max}$, residual tolerance τ.
OUTPUT: sparse vector $c^\star$.
(1) Initialize $r^{(0)} = y$, $S^{(0)} = \emptyset$, $c^{(0)} = 0$.
(2) For $n = 0, 1, 2, \dots$ until stopping:
  (a) Select the most correlated atom

$$j_{n+1} = \operatorname*{argmax}_{1 \le j \le M} |\langle a_j, r^{(n)} \rangle|.$$

  (b) Update support: $S^{(n+1)} = S^{(n)} \cup \{j_{n+1}\}$.
  (c) Refit on the active set:

$$c^{(n+1)} = \operatorname{argmin} \left\{ \|Ac - y\|_2^2 : \operatorname{supp}(c) \subseteq S^{(n+1)} \right\}.$$

  (d) Update residual: $r^{(n+1)} = y - Ac^{(n+1)}$.
(3) Stop if $|S^{(n+1)}| = s_{\max}$ or $\|r^{(n+1)}\|_2 \le \tau$.

OMP should be one of the first algorithms the team implements. It is simple, interpretable, and surprisingly strong when the true model is extremely sparse.

Option B: ℓ^1 recovery and the LASSO family. The fundamental convex sparse recovery problems are

$$\begin{aligned}
\text{Basis Pursuit (BP):} \qquad & \min_{c \in \mathbb{R}^M} \|c\|_1 \quad \text{subject to} \quad Ac = y, \\
\text{Basis Pursuit Denoising (BPDN):} \qquad & \min_{c \in \mathbb{R}^M} \|c\|_1 \quad \text{subject to} \quad \|Ac - y\|_2 \le \varepsilon, \\
\text{LASSO:} \qquad & \min_{c \in \mathbb{R}^M} \frac{1}{2}\|Ac - y\|_2^2 + \lambda \|c\|_1.
\end{aligned}$$

The ℓ^1 norm is the classical convex surrogate for ℓ^0 sparsity. Theoretical guarantees and efficient algorithms are both very mature in this setting [5, 10, 12, 16].

The proximal map of the ℓ^1 norm is the soft-thresholding operator

$$\mathcal{S}_\tau(v)_j = \operatorname{sign}(v_j)\max\{|v_j| - \tau, 0\},$$

which is one of the great little formulas of modern optimization. It turns a difficult-looking nonsmooth problem into a simple iteration.

Recommended algorithms.

- ISTA and FISTA for a general, reliable baseline [1].
- Coordinate descent for explicit moderate-size dictionaries [13].
- ADMM for robust splitting and easy constraint handling [3].
- Homotopy/LARS when one wants an entire regularization path for small-to-medium problems [11].
- SPGL1-style spectral projected methods for BPDN if the team later wants to handle large sparse problems [18].

Algorithm (FISTA for LASSO)
INPUT: A, y, regularization parameter $\lambda > 0$, Lipschitz constant $L \geq \|A^\top A\|_2$.
OUTPUT: approximate minimizer of $\frac{1}{2}\|Ac - y\|_2^2 + \lambda\|c\|_1$.

(1) Initialize $c^{(0)} = z^{(0)} \in \mathbb{R}^M$ and $t_0 = 1$.
(2) For $n = 0, 1, 2, \ldots$ until stopping:
  (a) Gradient step plus proximal map:

$$c^{(n+1)} = \mathcal{S}_{\lambda/L}\left(z^{(n)} - \frac{1}{L}A^\top(Az^{(n)} - y)\right).$$

  (b) Momentum update:

$$t_{n+1} = \frac{1 + \sqrt{1 + 4t_n^2}}{2},$$
$$z^{(n+1)} = c^{(n+1)} + \frac{t_n - 1}{t_{n+1}}\left(c^{(n+1)} - c^{(n)}\right).$$

(3) Stop when the relative objective decrease, support, and residual all stabilize.

A good undergraduate habit is to combine LASSO with a debiasing refit. Once a support S has been identified from the ℓ^1 solution, solve least squares on S. This typically improves coefficient accuracy substantially.

Option C: Weighted ℓ^1 and reweighted ℓ^1. A very natural extension is

$$\min_{c\in\mathbb{R}^M} \frac{1}{2}\|Ac - y\|_2^2 + \lambda\sum_{j=1}^M w_j|c_j|,$$

where $w_j > 0$ are user-chosen or iteratively updated weights. This is especially attractive in symbolic discovery because it allows us to penalize complicated atoms more strongly than simple ones. For example, if a dictionary contains both low-degree polynomials and more exotic terms, one may deliberately impose a complexity bias by taking larger weights on high-degree or transcendental atoms.

A standard reweighted scheme chooses

$$w_j^{(n+1)} = \frac{1}{|c_j^{(n)}| + \delta}$$

for some small $\delta > 0$, thereby approximating ℓ^0 behavior more aggressively than ordinary LASSO [6]. The proximal step remains explicit:

$$\left(\mathcal{S}_\tau^{(w)}(v)\right)_j = \operatorname{sign}(v_j)\max\{|v_j| - \tau w_j, 0\}.$$

This option should be among the first advanced extensions the team explores because it is easy to code once FISTA or ADMM is already implemented.

Option D: Elastic net for correlated dictionaries. When a dictionary contains highly correlated atoms, pure ℓ^1 regularization may behave erratically, selecting one of several near-duplicates almost arbitrarily. The elastic net addresses this by combining ℓ^1 and ℓ^2 penalties:

$$\min_{c\in\mathbb{R}^M} \frac{1}{2}\|Ac - y\|_2^2 + \lambda_1\|c\|_1 + \frac{\lambda_2}{2}\|c\|_2^2.$$

The extra ridge term stabilizes the fit and can improve performance substantially in wide, correlated libraries [20]. This is particularly relevant for mixed symbolic dictionaries where polynomial, rational, and transcendental atoms may shadow one another.

For coordinate descent with standardized columns, the update has the closed form

$$c_j \leftarrow \frac{1}{\|a_j\|_2^2 + \lambda_2} \mathcal{S}_{\lambda_1}\left(a_j^\top \left(y - \sum_{k \neq j} a_k c_k\right)\right).$$

Students should learn this model because it is one of the most practical regularizers in the presence of multicollinearity.

Option E: Group sparsity and sparse group sparsity. Many symbolic libraries have a natural block structure. For example, all Chebyshev coefficients of a fixed family may form a group; all atoms involving the same transcendental seed may form another; all coefficients associated with a fixed power of a parameter in a multivariate expansion may form yet another. In such cases it is natural to penalize groups rather than individual coefficients. The group LASSO solves

$$\min_{c \in \mathbb{R}^M} \frac{1}{2}\|Ac - y\|_2^2 + \lambda \sum_{g=1}^{G} \omega_g \|c_{G_g}\|_2,$$

where $\{G_g\}_{g=1}^G$ partitions the coefficient indices and $\omega_g > 0$ are weights [19].

If sparsity is desired both across and within groups, one may instead use the sparse group LASSO:

$$\min_{c \in \mathbb{R}^M} \frac{1}{2}\|Ac - y\|_2^2 + \lambda_1 \|c\|_1 + \lambda_2 \sum_{g=1}^{G} \omega_g \|c_{G_g}\|_2.$$

The proximal map for a single group is the vector soft-thresholding rule

$$\mathcal{G}_\tau(v) = \left(1 - \frac{\tau}{\|v\|_2}\right)_+ v,$$

with the convention $\mathcal{G}_\tau(0) = 0$. These models are ideal for multivariate Taylor expansions, parametric ODE families, and recurrence problems where an entire structural pattern may turn on or off together.

Option F: Analysis sparsity and total variation. Not every problem should be modeled as sparse coefficients in a synthesis dictionary. Sometimes the object of interest is not sparse, but a transformed version of it is. This leads to analysis sparsity models of the form

$$\min_{c \in \mathbb{R}^M} \frac{1}{2}\|Ac - y\|_2^2 + \lambda \|Bc\|_1,$$

where B is a discrete derivative, difference matrix, wavelet transform, or some other analysis operator. If B is the first-difference matrix, this becomes a fused LASSO or total variation-type penalty. Such models are useful when one expects piecewise smoothness, coefficient clustering, or abrupt regime transitions [7, 14].

This option is especially relevant for asymptotic discovery. Suppose a coefficient profile changes behavior across scales, or a learned relation is locally simple in one parameter regime and different elsewhere. Then TV-type penalties may reveal that structure more clearly than ordinary coefficient sparsity.

Option G: Nonconvex ℓ^p regularization for $0 < p < 1$. A more aggressive sparsity model is

$$\min_{c \in \mathbb{R}^M} \frac{1}{2}\|Ac - y\|_2^2 + \lambda \sum_{j=1}^{M} (|c_j|^2 + \varepsilon^2)^{p/2}, \qquad 0 < p < 1,$$

with a small smoothing parameter $\varepsilon > 0$. This is nonconvex and therefore more delicate, but it often produces much sparser solutions than ℓ^1 methods [8, 9]. Because the objective may possess multiple local minima, this option is best treated as an advanced track after the convex infrastructure is reliable.

A standard majorization/IRLS idea uses weights

$$w_j^{(n)} = (|c_j^{(n)}|^2 + \varepsilon^2)^{\frac{p}{2}-1}$$

and solves at each step a weighted ridge problem of the form

$$c^{(n+1)} = \operatorname*{argmin}_c \left\{ \frac{1}{2}\|Ac - y\|_2^2 + \frac{\lambda p}{2} \sum_{j=1}^{M} w_j^{(n)} c_j^2 \right\}.$$

The coefficients with small magnitude receive larger weights and are more strongly suppressed. It is a rather elegant machine: instead of directly solving a nasty nonconvex problem, one repeatedly solves a sequence of very civilized quadratic ones.

Option H: Robust losses with sparse regularization. When the numerical oracle occasionally produces outliers—for example because a quadrature routine struggles near a singularity, or because arithmetic counting functions fluctuate unpredictably—the squared loss may be too sensitive. Then one may consider

$$\min_{c\in\mathbb{R}^M} \sum_{k=1}^{N} \rho_\delta((Ac-y)_k) + \lambda\|c\|_1,$$

where ρ_δ is the Huber loss, or

$$\min_{c\in\mathbb{R}^M} \|Ac-y\|_1 + \lambda\|c\|_1.$$

These robust models are appropriate for difficult asymptotic or number-theoretic data sets where a few observations may behave badly. They also teach students an important lesson: if the data are awkward, change the loss, not just the solver.

Option I: Low-rank structure and nuclear norm regularization. Some discovery problems produce coefficient arrays rather than vectors. For example, when learning a bivariate expansion

$$u(a,t) \approx \sum_{m=0}^{M}\sum_{n=0}^{N} C_{mn} a^m t^n,$$

one may find that the coefficient matrix C is not only sparse but also approximately low-rank. A convex surrogate for rank is the nuclear norm. One may then consider

$$\min_{C} \frac{1}{2}\|\mathcal{A}(C)-y\|_2^2 + \lambda_*\|C\|_* + \lambda_1\|C\|_1.$$

Here $\mathcal{A}$ maps the matrix of coefficients to the sampled observations. The proximal map of the nuclear norm is singular value thresholding: if $X = U\,\mathrm{diag}(\sigma)V^\top$, then

$$\mathrm{prox}_{\tau\|\cdot\|_*}(X) = U\,\mathrm{diag}((\sigma_i-\tau)_+)V^\top.$$

This option is likely too advanced for the first round of students, but it is an excellent direction for a follow-up cohort because low-rank plus sparse structure appears naturally in separable formulas and parameterized families.

Optimization Methodologies the Students Should Learn Seriously

This section states plainly which optimization methodologies deserve priority in training. The guiding principle is that students should learn methods that are both mathematically principled and genuinely useful in research practice.

First, the linear algebra backbone. Before touching a single sparse regularizer, every student should master the following.

- Solve least squares by QR and SVD, not by forming $(A^\top A)^{-1}A^\top y$.
- Normalize columns and rescale inputs to natural domains such as $[-1,1]$.
- Compute singular values, condition numbers, and coherence.
- Use LSQR or conjugate-gradient-type inner solvers when matrices are too large or are applied matrix-free.
- Debias every sparse fit by least-squares refitting on the recovered support whenever the support is well-conditioned.

Without these habits, fancy regularization merely decorates bad numerics.

Proximal gradient as the baseline language. A huge class of problems takes the form

$$\min_{c\in\mathbb{R}^M} F(c) = f(c) + g(c),$$

where f is smooth and convex with Lipschitz gradient and g is convex but nonsmooth with an easy proximal map. The proximal operator is defined by

$$\mathrm{prox}_{\tau g}(v) = \operatorname*{argmin}_x \left\{ \frac{1}{2\tau}\|x-v\|_2^2 + g(x) \right\}.$$

Proximal gradient then updates by

$$c^{(n+1)} = \mathrm{prox}_{\tau g}\big(c^{(n)} - \tau\nabla f(c^{(n)})\big).$$

For LASSO, one has $f(c) = \frac{1}{2}\|Ac-y\|_2^2$ and $g(c) = \lambda\|c\|_1$, so the proximal map is soft thresholding.

This framework should be taught explicitly because it unifies a remarkable number of models. Once students understand proximal gradient and FISTA, they understand the computational heart of modern convex sparse optimization.

Coordinate descent for explicit regression libraries. When the dictionary is explicit and moderate in size, coordinate descent is one of the best practical choices for LASSO and elastic net. It is simple, warm-start friendly, and extremely effective for traversing a regularization path [13]. It also teaches students that optimization may exploit separability in a very direct way.

For normalized columns and the LASSO objective,

$$\min_c \frac{1}{2}\|Ac - y\|_2^2 + \lambda\|c\|_1,$$

the j-th coordinate update has the form

$$c_j \leftarrow \mathcal{S}_\lambda\left(a_j^\top\Big(y - \sum_{k\neq j} a_k c_k\Big)\right).$$

If the columns are not normalized, one divides by $\|a_j\|_2^2$ accordingly. Coordinate descent should be a core tool whenever the dictionary is not absurdly large.

ADMM for composite models and constrained problems. The alternating direction method of multipliers (ADMM) is one of the most useful splitting methods students can learn [3]. For the LASSO, write

$$\min_{c,z} \frac{1}{2}\|Ac - y\|_2^2 + \lambda\|z\|_1 \qquad \text{subject to} \qquad c = z.$$

The scaled ADMM iterations are

$$\begin{aligned} c^{(n+1)} &= (A^\top A + \rho I)^{-1}\big(A^\top y + \rho(z^{(n)} - u^{(n)})\big),\\ z^{(n+1)} &= \mathcal{S}_{\lambda/\rho}\big(c^{(n+1)} + u^{(n)}\big),\\ u^{(n+1)} &= u^{(n)} + c^{(n+1)} - z^{(n+1)}. \end{aligned}$$

Students should also learn the primal and dual residual stopping criteria

$$r^{(n+1)} = c^{(n+1)} - z^{(n+1)}, \qquad s^{(n+1)} = \rho\big(z^{(n+1)} - z^{(n)}\big).$$

ADMM is worth teaching because it extends almost mechanically to weighted ℓ^1, group sparsity, TV penalties, and constrained formulations.

Algorithm (ADMM for LASSO)
INPUT: A, y, $\lambda > 0$, penalty parameter $\rho > 0$.
OUTPUT: approximate minimizer of $\frac{1}{2}\|Ac - y\|_2^2 + \lambda\|c\|_1$.

(1) Initialize $c^{(0)}, z^{(0)}, u^{(0)}$.
(2) For $n = 0, 1, 2, \ldots$ until stopping:
  (a) Solve the linear system
  $$(A^\top A + \rho I)c^{(n+1)} = A^\top y + \rho(z^{(n)} - u^{(n)}).$$
  (b) Soft-threshold:
  $$z^{(n+1)} = \mathcal{S}_{\lambda/\rho}(c^{(n+1)} + u^{(n)}).$$
  (c) Dual ascent:
  $$u^{(n+1)} = u^{(n)} + c^{(n+1)} - z^{(n+1)}.$$
(3) Stop when both primal and dual residuals are sufficiently small.

Greedy and hard-thresholding methods for exact sparsity. Applied mathematicians take OMP, CoSaMP, IHT, and HTP seriously because these methods are cheap, interpretable, and often excellent when the target is truly sparse. They should not replace convex methods entirely, but they deserve equal footing in student training. In problems like the circle area law, where the true model is literally one-sparse in the right basis, greedy methods are often hard to beat.

The hard-thresholding operator that keeps the s largest-magnitude entries of a vector will be denoted by $\mathcal{H}_s$. Then IHT takes the form

$$c^{(n+1)} = \mathcal{H}_s\left(c^{(n)} + \alpha A^\top(y - Ac^{(n)})\right),$$

for a stepsize $\alpha > 0$. HTP augments this with a refit on the selected support:

$$S^{(n+1)} = \operatorname{supp}\left(\mathcal{H}_s\left(c^{(n)} + \alpha A^\top (y - Ac^{(n)})\right)\right),$$
$$c^{(n+1)} = \operatorname{argmin}\left\{\|Ac - y\|_2^2 : \operatorname{supp}(c) \subseteq S^{(n+1)}\right\}.$$

These are wonderfully close in spirit to the original LSSR pipeline, which makes them especially suitable for the first generation of students.

IRLS, MM, and reweighted methods for advanced sparsity promotion. Iteratively reweighted least squares (IRLS) should be taught after students understand least squares and LASSO. It is one of the prettiest bridges between smooth linear algebra and nonconvex sparsity promotion [9]. Even when used only for reweighted ℓ^1, it teaches the important idea of solving a hard problem through a sequence of easier surrogates.

Algorithm (IRLS for smoothed ℓ^p regularization)
INPUT: A, y, $\lambda > 0$, exponent $0 < p \le 1$, smoothing parameter $\varepsilon > 0$.
OUTPUT: approximate sparse minimizer.
(1) Initialize $c^{(0)}$.
(2) For $n = 0, 1, 2, \ldots$ until stopping:
  (a) Form weights
$$w_j^{(n)} = (|c_j^{(n)}|^2 + \varepsilon^2)^{\frac{p}{2}-1}.$$
  (b) Solve the weighted quadratic problem
$$c^{(n+1)} = \operatorname*{argmin}_c \left\{\frac{1}{2}\|Ac - y\|_2^2 + \frac{\lambda p}{2}\sum_{j=1}^{M} w_j^{(n)} c_j^2\right\}.$$
(3) Decrease ε cautiously if continuation is desired.

The inner solve is just a weighted ridge problem, so the method retains a strong linear algebra flavor. It is an excellent advanced project because it exposes students to the subtle tradeoff between better sparsity and loss of convexity.

Primal–dual and split Bregman methods for analysis sparsity. When the regularizer involves Bc rather than c itself, proximal gradient may become inconvenient. Then primal–dual hybrid gradient (PDHG) or split Bregman/ADMM-type methods are the right tools [7, 14]. Students do not need to master all of this at once, but they should learn that TV and fused LASSO problems are not exotic; they simply require the right splitting.

For example, one may rewrite

$$\min_c \frac{1}{2}\|Ac - y\|_2^2 + \lambda\|d\|_1 \qquad \text{subject to} \qquad d = Bc,$$

and then apply ADMM to (c, d). The c-update becomes a linear solve involving $A^\top A + \rho B^\top B$, while the d-update is once again soft-thresholding. This is quite manageable once students already know the LASSO version of ADMM.

Interior-point and modeling-language solvers as benchmarks, not crutches. Students should be allowed to use CVX, CVXPY, or other modeling tools as gold-standard benchmark solvers on small instances. This is useful for debugging and for checking whether a custom implementation is behaving correctly. But these tools should not replace algorithmic understanding. The goal is not to worship a convex black box as if it were a benevolent oracle. The goal is to know why the solver works, what linear systems it is really solving, and when a lighter bespoke method is preferable.

Precise Regularization Paths and Parameter Selection

Sparse optimization is inseparable from parameter tuning. If this part is sloppy, the whole enterprise becomes numerology.

Selecting λ, s, and ε. For the LASSO with normalized columns, the critical parameter

$$\lambda_{\max} = \|A^\top y\|_\infty$$

is the smallest value for which the zero vector solves the problem. A robust strategy is to compute a decreasing sequence

$$\lambda_{\max} = \lambda_0 > \lambda_1 > \cdots > \lambda_K > 0,$$

use warm starts along this path, and choose the final parameter by validation error, support stability, or a discrepancy principle based on an estimate of the noise level.

For BPDN, the constraint parameter ε should reflect the numerical error in the data generation process. For instance, if a quadrature rule and ODE solver are both accurate to a prescribed tolerance, then students should propagate that tolerance into a realistic estimate for $\|e\|_2$.

For ℓ^0 methods, the sparsity level s should not be chosen by wishful thinking. Instead, students should inspect support stability, validation performance, and the conditioning of the selected submatrices as s varies.

Debiasing is not optional. Whenever a sparse model is selected by LASSO, OMP, IHT, HTP, or reweighted methods, the team should perform a post-selection least-squares fit on the active support provided that the support is well-conditioned. Regularization is mainly for support discovery and stability. Once the support is accepted, the coefficients should be estimated by the best linear algebra available on that support.

A practical stopping criterion hierarchy. Students should monitor the following simultaneously:

(1) relative objective decrease,
(2) residual norm decrease,
(3) support stabilization,
(4) KKT residuals when available,
(5) validation error.

No single criterion is always trustworthy by itself. Optimization methods are clever, but they are not mind readers.

A Technical Research Roadmap for the Undergraduate Team

The following is a serious but realistic semester-length roadmap. It is designed so that weaker students can contribute at the level of implementation and diagnostics, while stronger students can push toward theory, structured models, and proofs.

Phase I: Numerical linear algebra and conditioning (Weeks 1–2).

(1) Implement least squares using QR and SVD.
(2) Reproduce the circle-area problem in monomial, Legendre, and Chebyshev bases.
(3) Normalize columns and compare condition numbers and coherence.
(4) Write a short note explaining why normal equations are avoided in serious computation.

Deliverable. A reproducible notebook that compares basis choice, condition numbers, support quality, and coefficient error on the warm-up problem.

Phase II: Greedy sparse recovery (Weeks 3–4).

(1) Implement OMP and HTP from scratch.
(2) Compare them against the original LSSR threshold-and-refit strategy.
(3) Study exact support recovery on the circle-area law and on short Maclaurin expansions such as e^{at}.
(4) Compute the support recovery rate across many random training sets.

Deliverable. A benchmark table comparing OMP, HTP, and LSSR across sample size, basis type, and noise level.

Phase III: Convex sparsity and proximal algorithms (Weeks 5–6).

(1) Implement ISTA and FISTA for the LASSO.
(2) Implement ADMM for the LASSO and BPDN.
(3) Validate all solvers against a modeling-language benchmark on small problems.
(4) Explore regularization paths and warm starts.

Deliverable. A report showing objective curves, support recovery, and debiased coefficient error for ISTA, FISTA, and ADMM.

Phase IV: Weighted models and correlated libraries (Weeks 7–8).

(1) Introduce weighted ℓ^1 and reweighted ℓ^1.
(2) Build a mixed dictionary containing orthogonal polynomials, logarithmic atoms, reciprocal powers, and trigonometric terms where appropriate.
(3) Compare LASSO against elastic net on highly correlated libraries.
(4) Introduce symbolic complexity weights to penalize overly complicated atoms.

Deliverable. A write-up showing how weighted penalties help recover simpler formulas from a wider mixed dictionary.

Phase V: Structured sparsity for multivariate problems (Weeks 9–10).

(1) Implement group LASSO or sparse group LASSO for coefficient blocks.
(2) Apply these methods to bivariate expansions such as e^{at} or to parametric integral families.
(3) Study low-rank structure in coefficient matrices, at least experimentally.

Deliverable. A multivariate example where group structure improves interpretability over plain coefficient-wise sparsity.

Phase VI: Advanced track (Weeks 11–12). Students who remain on track may choose one of the following.

- IRLS for nonconvex ℓ^p sparsity.
- TV or fused LASSO for analysis sparsity.
- Robust sparse regression with Huber or ℓ^1 loss.
- Low-rank plus sparse recovery for coefficient matrices.

Deliverable. A short technical note that includes algorithm derivation, implementation details, and a careful validation study.

Problem Bank Mapped to Compressed Sensing Models

Below is a mathematically natural mapping between problem types and compressed sensing tools.

Target problem	**Natural dictionary**	**Suggested regularization**	**Best first solvers**
Area of a circle	monomials, Legendre, Chebyshev	ℓ^0, LASSO, weighted ℓ^1	OMP, HTP, FISTA, ADMM
e^{at} from ODE data	tensor monomials in (a,t), grouped diagonals	group sparsity, sparse group sparsity	block proximal gradient, ADMM
$\arcsin(x)$ and $\log(x)$	power series, Chebyshev, rational atoms	weighted ℓ^1, elastic net	FISTA, coordinate descent
Semicircle and Beta integrals	polynomial or parameterized basis families	weighted ℓ^1, group sparsity	ADMM, proximal gradient
Asymptotic laws	mixed dictionaries with logs and powers	weighted ℓ^1, TV, robust loss	ADMM, PDHG, IRLS
Recurrence discovery	shifted sequences and block libraries	group sparsity, ℓ^0 constraints	OMP, HTP, block descent
Coefficient arrays in multivariate formulas	tensor-product matrix libraries	sparse + low-rank	proximal splitting, ADMM
Noisy arithmetic data	mixed symbolic atoms	robust sparse regression	ADMM, proximal gradient, IRLS

Standards of Evidence: When a Numerical Pattern Becomes Mathematics

The purpose of this project is not to manufacture seductive numerical coincidences. The goal is to turn computation into credible mathematical conjecture and, whenever possible, proof. For this reason, every claimed discovery should pass through the following filter.

(1) **Out-of-sample validation.** The recovered law must generalize beyond the training points.
(2) **Support stability.** The support must persist under re-sampling, modest perturbations of numerical tolerances, and reasonable changes in basis resolution.
(3) **Debiased coefficient accuracy.** Once the support is fixed, the refitted coefficients should be stable to high precision.
(4) **Exact identification of constants.** Floating-point numbers that appear to be familiar constants should be tested by rational reconstruction, continued fractions, or integer-relation methods such as PSLQ, but only after the numerical evidence is stable.
(5) **Proof.** The final step is not optional. A sparse fit is a conjecture generator, not a theorem.

For a compressible target, students should explicitly distinguish between symbolic recovery and accurate approximation. Some laws truly are sparse in the chosen dictionary; others are merely well approximated by a few terms. Both are mathematically interesting, but they are not the same claim.

What the Team Should Master First

If we want students to learn the best methodologies without drowning them in every possible variant, then the order of mastery should be the following.

(1) **Nonnegotiable core:** QR/SVD least squares, basis scaling, column normalization, support refitting.
(2) **First sparse algorithms:** OMP and HTP.

(3) **First convex algorithms:** ISTA and FISTA for the LASSO.
(4) **First splitting method:** ADMM for LASSO and BPDN.
(5) **First structured extension:** weighted ℓ^1 and elastic net.
(6) **Second structured extension:** group sparsity.
(7) **Advanced methods:** IRLS for nonconvex ℓ^p, TV/fused LASSO, robust losses, low-rank penalties.

This hierarchy gives students tools that are immediately useful, mathematically principled, and broad enough to cover nearly every discovery problem proposed in the main project. The compressed sensing world is large, but it does not need to be served all at once like a buffet at a conference hotel. A well-paced progression will do much more good.

Concluding Technical Vision

Compressed sensing does not replace the original spirit of *Learning Mathematics Through the Machine*; it sharpens it. The original LSSR framework already demonstrated that a machine can rediscover a law such as $g(r) = \pi r^2$ from numerical data. Compressed sensing expands that framework to settings where the dictionary is wider, the structure is subtler, and the need for principled sparsity is unavoidable. It also gives the undergraduate team a genuine research-level toolbox whose ingredients are central to modern applied mathematics: approximation theory, inverse problems, optimization, numerical linear algebra, and statistical validation.

The guiding philosophy remains unchanged. We are not asking machines to replace mathematical thought. We are building computational instruments that help students locate the right conjectures, organize evidence, and expose hidden structure. Once the sparse coefficients settle and the support stops wobbling, the mathematics begins in earnest. That is where the proof takes over and the machine finally becomes what it should have been all along: not an oracle, but a very patient assistant with excellent linear algebra.

References

[1] A. Beck and M. Teboulle, *A fast iterative shrinkage-thresholding algorithm for linear inverse problems*, SIAM Journal on Imaging Sciences **2** (2009), 183–202.
[2] T. Blumensath and M. E. Davies, *Iterative hard thresholding for compressed sensing*, Applied and Computational Harmonic Analysis **27** (2009), 265–274.
[3] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, *Distributed optimization and statistical learning via the alternating direction method of multipliers*, Foundations and Trends in Machine Learning **3** (2011), 1–122.
[4] A. M. Bruckstein, D. L. Donoho, and M. Elad, *From sparse solutions of systems of equations to sparse modeling of signals and images*, SIAM Review **51** (2009), 34–81.
[5] E. J. Candès, J. Romberg, and T. Tao, *Stable signal recovery from incomplete and inaccurate measurements*, Communications on Pure and Applied Mathematics **59** (2006), 1207–1223.
[6] E. J. Candès, M. B. Wakin, and S. Boyd, *Enhancing sparsity by reweighted ℓ^1 minimization*, Journal of Fourier Analysis and Applications **14** (2008), 877–905.
[7] A. Chambolle and T. Pock, *A first-order primal-dual algorithm for convex problems with applications to imaging*, Journal of Mathematical Imaging and Vision **40** (2011), 120–145.
[8] R. Chartrand, *Exact reconstruction of sparse signals via nonconvex minimization*, IEEE Signal Processing Letters **14** (2007), 707–710.
[9] I. Daubechies, R. DeVore, M. Fornasier, and C. S. Güntürk, *Iteratively reweighted least squares minimization for sparse recovery*, Communications on Pure and Applied Mathematics **63** (2010), 1–38.
[10] D. L. Donoho, *Compressed sensing*, IEEE Transactions on Information Theory **52** (2006), 1289–1306.
[11] B. Efron, T. Hastie, I. Johnstone, and R. Tibshirani, *Least angle regression*, The Annals of Statistics **32** (2004), 407–499.
[12] S. Foucart and H. Rauhut, *A Mathematical Introduction to Compressive Sensing*, Birkhäuser, New York, 2013.
[13] J. Friedman, T. Hastie, and R. Tibshirani, *Regularization paths for generalized linear models via coordinate descent*, Journal of Statistical Software **33** (2010), 1–22.
[14] T. Goldstein and S. Osher, *The split Bregman method for ℓ^1-regularized problems*, SIAM Journal on Imaging Sciences **2** (2009), 323–343.
[15] D. Needell and J. A. Tropp, *CoSaMP: Iterative signal recovery from incomplete and inaccurate samples*, Applied and Computational Harmonic Analysis **26** (2009), 301–321.
[16] R. Tibshirani, *Regression shrinkage and selection via the lasso*, Journal of the Royal Statistical Society, Series B **58** (1996), 267–288.
[17] J. A. Tropp and A. C. Gilbert, *Signal recovery from random measurements via orthogonal matching pursuit*, IEEE Transactions on Information Theory **53** (2007), 4655–4666.
[18] E. van den Berg and M. P. Friedlander, *Probing the Pareto frontier for basis pursuit solutions*, SIAM Journal on Scientific Computing **31** (2008/09), 890–912.
[19] M. Yuan and Y. Lin, *Model selection and estimation in regression with grouped variables*, Journal of the Royal Statistical Society, Series B **68** (2006), 49–67.
[20] H. Zou and T. Hastie, *Regularization and variable selection via the elastic net*, Journal of the Royal Statistical Society, Series B **67** (2005), 301–320.